

Executable Specification

Lập trình ở tầng ý định — Khi Spec trở thành ngôn ngữ máy hiểu được

Chương này xây dựng toàn bộ công cụ để viết Executable Spec: từ triết lý thiết kế, cấu trúc 8 thành phần, đến EARS Notation — ngôn ngữ đặc tả yêu cầu được thiết kế riêng để loại bỏ sự mơ hồ.

01

5.1

Spec là Giao diện

02

5.2

8 Thành phần

03

5.3

EARS Notation

04

5.4

Spec Depth

05

5.5

SDD Workflow

Giới thiệu chương

Có một nghịch lý thú vị trong thực tế phát triển phần mềm với AI: chúng ta trao cho AI khả năng viết code phức tạp, nhưng lại không dành đủ thời gian để nói với AI chính xác điều mình muốn. Kết quả là AI "đoán" — đôi khi đúng, nhiều khi sai, và luôn luôn không nhất quán.

Executable Specification (Spec có thể thực thi) là câu trả lời cho nghịch lý đó. Đây không phải tài liệu Word mà một người đọc rồi diễn giải theo cách riêng. Đây là một hệ thống ngôn ngữ có cấu trúc chặt chẽ — đủ rõ ràng để AI biến nó thành code mà không cần đoán, đủ có thể đọc để con người review và maintain.

- ❏ **Mục tiêu học tập** Hiểu Executable Spec là "interface" giữa tư duy con người và năng lực thực thi của AI. Viết spec đầy đủ 8 thành phần, đặc biệt biết khi nào cần thêm "Out of Scope". Thuần thực EARS Notation — 5 patterns với Cheat Sheet tham khảo nhanh. Áp dụng đúng mức độ spec (Sketch / Detailed / Formal) theo Risk-Complexity Matrix. Thực hành quy trình SDD 4 bước: Draft → AI Review → Finalize → Generate. Nhận diện và tránh 5 Anti-patterns khiến AI hallucinate.

Nội dung Chương 5

5.1 · Spec là "Giao diện"

Interface giữa Người và Máy · PRD truyền thống vs Executable Spec · Precision & Hallucination · Tách What khỏi How

5.2 · 8 Thành phần Cốt lõi

Context & Goal · Actors · Functional · NFR
· Data · Error · Acceptance · Out of Scope
· 5 Anti-patterns

5.3 · EARS Notation

5 patterns: Ubiquitous / Event / State / Optional / Unwanted · Cheat Sheet · SHALL/SHOULD/MAY · Kết hợp patterns

5.4 · Levels of Spec Depth

Risk × Complexity Matrix · Sketch / Detailed / Formal · State Diagram · Decision guide

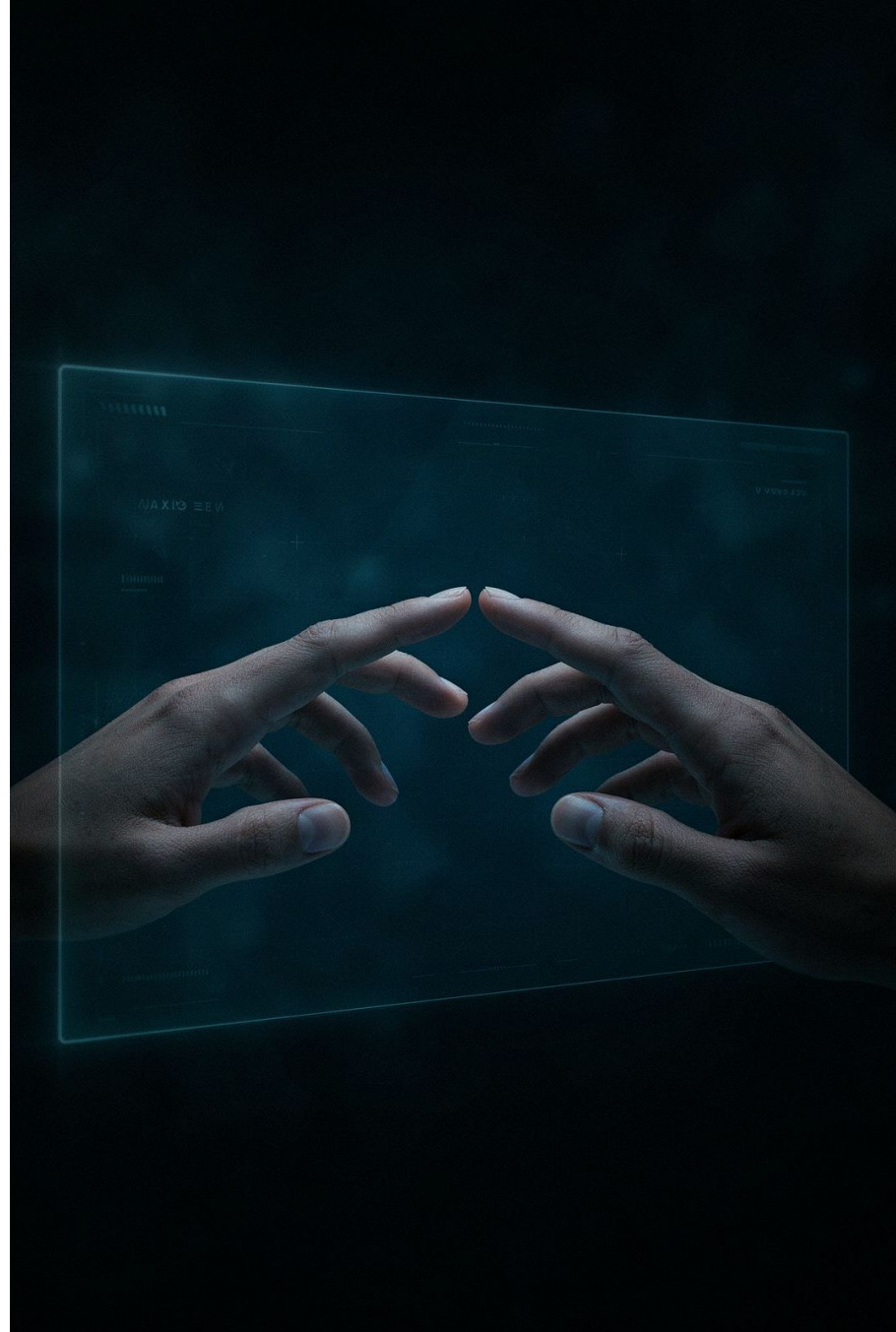
5.5 · Workflow

Draft → AI Review → Finalize Source of Truth → Generate Code/Test · kèm bài tập tổng hợp để luyện quy trình end-to-end.

5.1

Spec là "Giao diện"

Interface giữa Người và Máy



5.1 Spec là "Giao diện" — Interface giữa Người và Máy

Trong lập trình hướng đối tượng, Interface là hợp đồng: nó định nghĩa những gì một thành phần phải làm mà không quan tâm đến cách nó làm. Hai bên — người gọi và người thực thi — đều chỉ cần biết Interface, không cần biết chi tiết bên trong của nhau. Giao tiếp được tách biệt khỏi implementation.

Executable Specification đóng vai trò y hệt trong giao tiếp giữa con người và AI Agent. Spec là hợp đồng giữa "ý định của con người" và "năng lực thực thi của AI". Con người không cần biết AI sẽ sinh code ra sao. AI không cần đọc tâm trí con người. Cả hai gặp nhau tại Spec — một văn bản đủ chính xác để không cần giải thích thêm.

Bên trái Interface — Con người	Interface — SPEC.md	Bên phải Interface — AI Agent
Mang ý định, kinh nghiệm domain, judgment về bài toán cần giải quyết	Hợp đồng chính xác — zero ambiguity, executable, testable	Mang năng lực thực thi — generate code, test, không mệt mỏi

 **Key insight** Spec không phải tài liệu để đọc — Spec là chương trình. AI là compiler. Code là bytecode. Người sở hữu Spec chính là người sở hữu sản phẩm.

5.1.1 — Tại sao PRD truyền thống thất bại với AI

Vấn đề cốt lõi: PRD truyền thống được viết cho con người — người có thể suy luận, hỏi lại, và lấp đầy khoảng trống bằng common sense. AI không làm được điều đó: nó chỉ thực thi — và khi gặp mơ hồ, nó hallucinate.

Chiều cạnh	PRD truyền thống	Executable Specification
Người đọc mục tiêu	Con người — tự suy luận	AI Agent — phải thực thi trực tiếp
Độ rõ ràng (Precision)	Cho phép mơ hồ, con người lấp đầy	Zero ambiguity — thiếu logic = AI hallucinate
Ngôn ngữ	Ngôn ngữ tự nhiên, narrative	Ngôn ngữ có cấu trúc (EARS, BDD...)
Điều kiện biên	Ngụ ý hoặc bỏ qua	Explicit, được liệt kê đầy đủ
Phạm vi	Thường chỉ nói những gì làm	Bao gồm cả "Out of Scope" — những gì KHÔNG làm
Testability	Khó đo lường trực tiếp	Mỗi requirement → test case trực tiếp
Xử lý lỗi	Thường được để ngụ ý	Explicit — mọi failure path đều được đặc tả

5.1.2 Độ rõ ràng — Precision và vấn đề Hallucination

Đây là điểm mấu chốt mà nhiều kỹ sư bỏ qua khi lần đầu viết spec cho AI. Khi một developer con người gặp một yêu cầu mơ hồ, họ có ba lựa chọn: hỏi lại, suy luận từ context, hoặc tạm bỏ qua. Khi một AI Agent gặp yêu cầu mơ hồ, nó chỉ có một lựa chọn: đoán — và đưa đoán đó ra như thể nó là sự thật.

Hiện tượng này gọi là "confident hallucination" — AI không nói "tôi không chắc", nó nói "đây là cách tôi hiểu và đây là code tôi viết". Kết quả: code chạy được, pass basic tests, nhưng implement sai logic. Lỗi này cực kỳ nguy hiểm vì khó phát hiện hơn lỗi compile hay runtime.

✗ PRD truyền thống — Mơ hồ, nguy hiểm

"Hệ thống phải xử lý đăng nhập một cách thông minh, đảm bảo bảo mật và trải nghiệm người dùng tốt."

AI tự quyết: "thông minh" → ML model? · "bảo mật" → bcrypt? SHA256? · "trải nghiệm tốt" → auto-fill? biometric? → 3 developer, 3 implementation khác nhau.

✓ Executable Spec — Không còn chỗ để đoán

GIVEN người dùng nhập email + password hợp lệ WHEN nhấn "Đăng nhập" THEN: Hash bcrypt (cost=12), JWT RS256 expiry=15 phút, failed≥5 → lock 30 phút, return 200+token HOẶC 401+error_code.

Mọi quyết định đã được con người đưa ra. AI chỉ còn việc implement.

- 📌 **Nguyên tắc vàng** "If AI has to guess, you have already failed." — Mỗi điểm mơ hồ trong spec là một điểm AI có thể hallucinate. Rule of thumb: đọc lại spec và tự hỏi "câu này có thể hiểu theo nhiều cách không?" — Nếu có → viết lại. Làm cho nó có đúng MỘT cách hiểu duy nhất.

5.1.3 Spec như Interface — Tách biệt "Cái gì" (What) và "Như thế nào" (HOW)

Một trong những giá trị lớn nhất của Executable Spec là nó buộc bạn tách biệt hai câu hỏi hoàn toàn khác nhau: "Hệ thống phải làm gì?" (What) và "Hệ thống sẽ làm điều đó như thế nào?" (How). Spec chỉ trả lời câu hỏi What. How là trách nhiệm của AI implementation — và AI giỏi việc đó hơn bạn nghĩ, miễn là What đủ rõ ràng.

Sự tách biệt này mang lại hai lợi ích quan trọng. Thứ nhất, nó cho phép bạn thay đổi implementation mà không cần viết lại spec. Thứ hai, nó cho phép bạn test behavior thay vì test implementation — test rằng "kết quả đúng" thay vì "code chạy theo đúng cách này".

WHAT — Spec mô tả

- Password phải được hash trước khi lưu
- Hash algorithm: bcrypt, cost factor ≥ 12
- Login thất bại 5 lần $\rightarrow$ lock 30 phút
- Success $\rightarrow$ trả về JWT, TTL = 15 phút
- Error response phải include error_code

HOW — AI tự quyết định

- Import library nào để bcrypt
- Structure class/function như thế nào
- Error handler middleware hay try-catch
- Redis để lưu lock hay DB counter
- Middleware chain hay service layer

📌 **Design Principle** Spec tốt là Spec mà khi đọc xong, bạn có thể kiểm tra tính đúng đắn của implementation mà không cần xem code — chỉ cần chạy tests dựa trên Acceptance Criteria.



5.2

8 Thành phần Cốt lõi

Executable Spec · Cấu trúc đầy đủ

5.2 Cấu trúc Executable Spec — 8 Thành phần Cốt lõi

Một Executable Specification đầy đủ cần tám thành phần, mỗi thành phần giải quyết một khía cạnh riêng biệt của bài toán. Thiếu bất kỳ thành phần nào cũng tạo ra "lỗ hổng context" — những khoảng trống mà AI sẽ tự lấp đầy bằng các giả định không được kiểm soát.

#	Thành phần	Câu hỏi trả lời	Thiếu → AI làm gì?
1	Context & Goal	Tại sao feature này tồn tại?	Code "đúng kỹ thuật" nhưng sai bài toán
2	Actors & Roles	Ai tương tác? Với quyền gì?	Bỏ qua phân quyền, code cho một loại user
3	Functional Requirements	Hệ thống làm gì?	Implement theo hiểu biết default của model
4	Non-functional Requirements	Tốt đến mức nào?	Performance/security theo best guess
5	Data Model	Dữ liệu có cấu trúc gì?	Tự thiết kế schema — thường không phù hợp
6	Error Handling	Khi sai thì làm gì?	Happy path only — production sẽ crash
7	Acceptance Criteria	Định nghĩa "xong" là gì?	Không có target — tests sẽ yếu và thiếu
8	Out of Scope	Hệ thống KHÔNG làm gì?	"Nhiệt tình" thêm features không cần thiết

5.2.1 — Context & Goal · "Tại sao" (Why) trước khi "Cái gì" (What)

Thành phần này thường bị bỏ qua vì có cảm giác "không technical". Đây là sai lầm. AI model được train trên hàng triệu codebase — khi bạn cung cấp business context, model có thể align implementation với những pattern phổ biến nhất cho use case đó, thay vì chọn một pattern ngẫu nhiên.vvvv

Ví dụ — Context & Goal tốt (Password Reset Feature)

Context & Goal

****Business problem:**** Người dùng quên password thường phải liên hệ support mất 24-48 giờ.

Điều này tạo ra 340 support tickets/tháng.

****Feature goal:**** Cho phép người dùng tự reset password trong < 5 phút mà không cần human support.

****Success metric:**** Giảm support tickets liên quan đến password xuống 80%.

****Tech context:**** Đây là extension của auth service hiện tại.

Stack: Node.js 20, Express, PostgreSQL 16, Redis 7, SendGrid API.

Auth hiện tại dùng JWT (RS256). Password hash: bcrypt cost=12.

❏ **Tại sao business problem quan trọng?** Khi AI biết rằng feature này tạo ra để giảm 340 support tickets/tháng, nó sẽ ưu tiên reliability và user experience hơn là elegant architecture. Business context định hướng trade-off decisions của AI.

5.2.2–5.2.7 — Actors, Functional, NFR, Data, Error, Acceptance

5.2.2 Actors & Roles

Actors & Roles

Actor	Mô tả	Permissions
Registered User	Account hợp lệ	Khởi tạo reset, nhập token, đặt pass mới
System (Auto)	Email service	Gửi email, expire tokens tự động
Admin	Internal staff	Xem audit log, force-expire tokens

- **Actors KHÔNG trong scope:**
- Guest user (chưa có account)
 - OAuth users (không có password)
 - Service accounts (dùng API key)

5.2.3–5.2.7 — 5 điểm cốt lõi

- Functional: Dùng EARS Notation. Mỗi requirement là một câu có thể test được.
- Non-functional: Phải có số đo cụ thể. "Nhanh" là vô nghĩa. "P95 latency < 200ms" là executable.
- Data Model: Phải align với schema thực tế. Nếu DB đã có, paste relevant tables vào.
- Error Handling: Liệt kê TỪNG failure mode, không chỉ happy path.
- Acceptance Criteria: Viết theo format Given-When-Then. Nếu AI dùng criteria này để viết test, nó phải pass.

❑ **NFR Example** ❌ "Hệ thống phải nhanh" → ✅ "API response time: P95 < 150ms, P99 < 500ms dưới 1000 rps"

5.2.8 — Out of Scope · Ranh giới quan trọng nhất

Đây là thành phần ít phổ biến nhất nhưng lại có tác động lớn nhất đến chất lượng code sinh ra. AI model được train để "helpful" — nó có xu hướng thêm những gì nó nghĩ là hữu ích, ngay cả khi bạn không yêu cầu. Không có "Out of Scope" tường minh, AI sẽ tự quyết định biên giới của feature — và thường quyết định sai.

Tại sao AI "nhiệt tình quá mức"? AI học từ hàng triệu codebase chất lượng tốt — trong đó features thường đi kèm nhau. Khi bạn yêu cầu "login form", AI đã thấy hàng nghìn ví dụ có kèm: remember me, forgot password, social login, 2FA... và nó cho rằng đây là "complete implementation". Kết quả: codebase phình to, scope creep, tech debt từ code không được test kỹ.

Ví dụ — Out of Scope đầy đủ (Password Reset)

Out of Scope — Sprint hiện tại

KHÔNG thực hiện trong sprint này:

- 2FA/MFA trong quá trình reset (phase 2)
- Reset qua SMS/phone (chỉ email)
- Admin-triggered password reset (different flow)
- Password strength enforcer UI (backend validation chỉ)
- Remember device sau khi reset · Audit trail UI (chỉ logging)

Lý do loại trừ (giúp AI hiểu context, không chỉ tuân thủ):

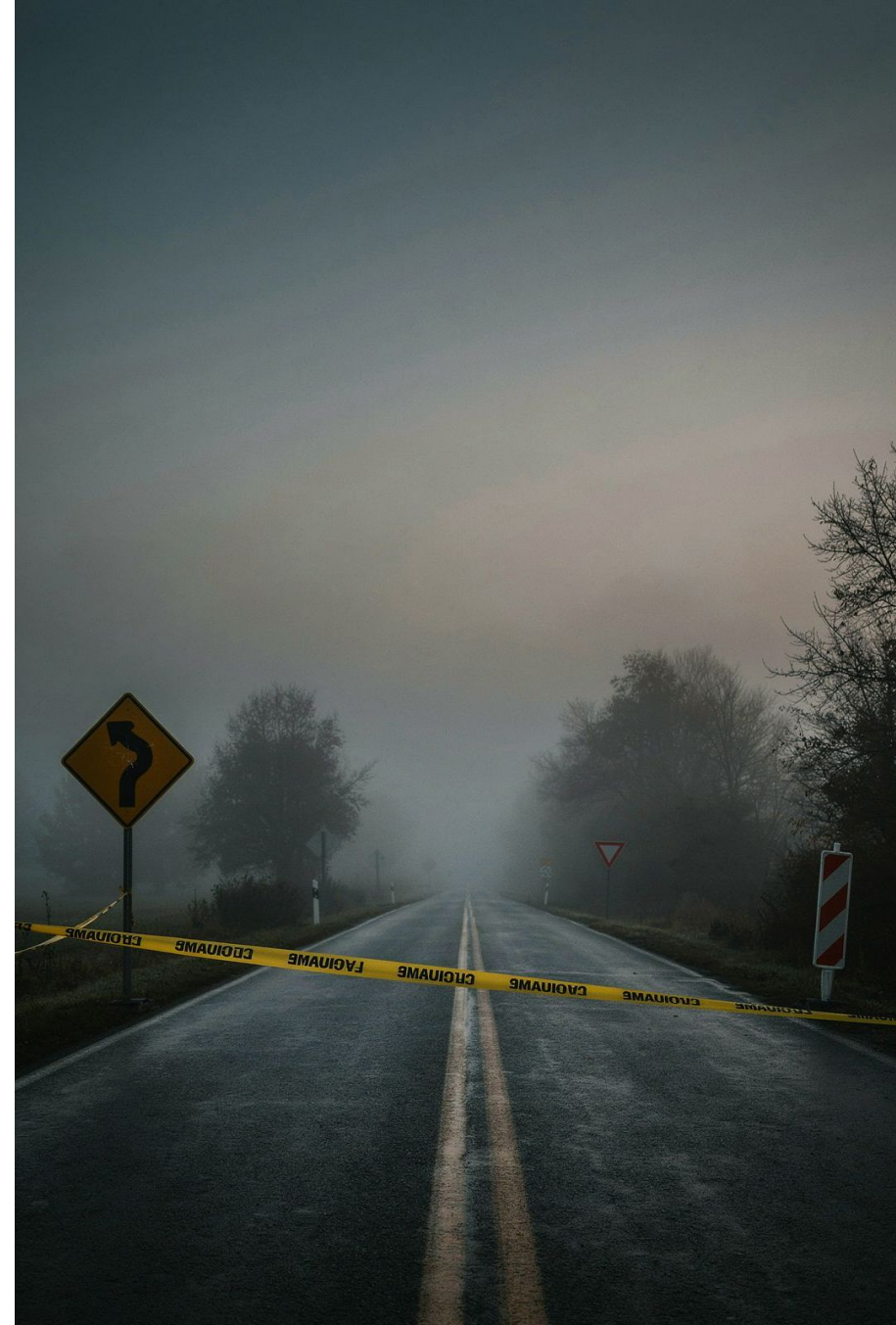
- 2FA: đang có epic riêng, tránh dependency
- SMS: SendGrid đã ký contract, chưa có SMS provider

Boundary conditions — AI không được tự quyết:

- KHÔNG tự thêm migration cho bảng users (schema đã lock)
- KHÔNG thay đổi JWT structure hiện tại
- KHÔNG implement email template — dùng SendGrid template ID đã có

5.2.9 — Anti-patterns · "Tử Huyệt" khi viết Spec cho AI

Sau đây là năm anti-pattern phổ biến nhất, mỗi loại đều được ghi nhận từ thực tế của các team đã áp dụng AI-assisted development. Nhận biết và tránh những lỗi này là kỹ năng phân biệt kỹ sư giỏi với kỹ sư trung bình trong kỷ nguyên AI.



Anti-pattern 1 — "The Magic Requirement"

Dùng những tính từ đánh giá mà không có tiêu chí đo lường. AI không có context để định nghĩa "nhanh", "thông minh", "mượt mà" — nó sẽ dùng định nghĩa của chính nó, thường là từ codebase training data phổ biến nhất.

✗ "Magic" Requirements

- "Giao diện phải mượt mà và thân thiện."
- "Hệ thống phải xử lý request nhanh chóng."
- "Thuật toán phải thông minh và tối ưu."

✓ Executable Equivalents

- "Animation transition: 200ms ease-in-out, 60fps."
- "API response: P95 < 150ms, P99 < 500ms."
- "Sorting: $O(n \log n)$ worst-case, stable sort."

Anti-pattern 2 — "The Contradiction"

Hai yêu cầu mâu thuẫn nhau mà không có cơ chế giải quyết. AI sẽ không báo lỗi — nó sẽ chọn một trong hai (thường là cái dễ implement hơn) và bỏ qua cái còn lại. Không có warning, không có error, chỉ có implementation sai.

✗ Contradiction — AI phải chọn một

- "Bảo mật cực cao: 2FA bắt buộc, session timeout 5 phút."
- "UX tốt: đăng nhập một chạm, nhớ session 30 ngày."

✓ Resolved — Con người đã quyết định

- "Đọc data: session JWT, không re-auth
- Ghi/sửa: re-enter password
- Xóa/billing: bắt buộc 2FA TOTP
- Nhớ device: tối đa 30 ngày, chỉ đọc"

Anti-patterns 3, 4, 5 (tiếp)

Anti-pattern 3 — "The Context Gap"

Quên không cung cấp thông tin về stack hiện tại, conventions đang dùng, hoặc constraints của môi trường. AI sẽ dùng default knowledge từ training — đôi khi đúng, thường là conflict với codebase thực tế.

✗ Context Gap

"Xây dựng DatePicker cho form booking." — AI sẽ chọn: react-datepicker? MUI? date-fns? Tailwind? useState? Formik? Zod? vi-VN hay English?

✓ Context Provided

"DatePicker cho booking. Tech: shadcn/ui v2, Tailwind CSS v4, React Hook Form v8, Zod schema, date-fns v4, locale vi-VN. Pattern: xem /src/components/forms/TimePicker.tsx"

Anti-pattern 4 — "The Implicit Assumption"

Những điều "hiển nhiên" với con người thường hoàn toàn không hiển nhiên với AI. Đặc biệt là các business rules ẩn, regulatory requirements, hay domain invariants.

- ❏ **Ví dụ** "Cho phép user cập nhật tài khoản bank." — Con người tự hiểu: phải verify, audit log, không sửa khi pending transaction. AI không biết những điều này.

Anti-pattern 5 — "The Moving Target"

Spec không có version, không có ownership, và thay đổi trong quá trình AI đang implement. Đây là recipe cho inconsistent codebase — code viết dựa trên spec v1 sẽ conflict với code viết dựa trên spec v2.

- ❏ **Giải pháp** SPEC.md — v1.3.0, Owner: @backend-lead, Status: APPROVED. Rule: Không modify spec đang trong sprint. Phát sinh → tạo addendum.

Checklist trước khi đưa Spec cho AI

Anti-pattern Checklist

- ❗ Không có từ nào là "thông minh", "tối ưu", "nhANH" mà không có số đo cụ thể
- ❗ Tất cả contradictions đã được resolve, trade-off đã được document
- ❗ Tech stack và conventions hiện tại đã được liệt kê đầy đủ
- ❗ Mọi business rule ẩn đã được viết tường minh
- ❗ Spec có version, có owner, và không thay đổi trong sprint
- ❗ "Out of Scope" đã được viết — không chỉ có những gì CẦN làm

5 Anti-patterns Summary

#	Tên	Dấu hiệu nhận biết
1	Magic Req.	Tính từ không có số đo: nhanh, thông minh, tốt
2	Contradiction	Hai yêu cầu xung đột, không có tie-breaker
3	Context Gap	Thiếu tech stack, library, existing patterns
4	Implicit Assume	Business rule, regulatory req. không viết ra
5	Moving Target	Spec không có version, thay đổi mid-sprint

📌 **Mindset** Viết spec như viết contract pháp lý — mọi điều cần tường minh, mọi ranh giới cần rõ ràng. AI là bên thực thi hợp đồng, không phải đối tác sáng tạo.

5.3

EARS Notation

Vũ khí Tối thượng Chống Mơ hồ



5.3 EARS Notation — Vũ khí Tối thượng Chống Mơ hồ

Năm 2009, Alistair Mavin và nhóm kỹ sư tại Rolls-Royce (hàng không vũ trụ, không phải ô tô) đang đối mặt với một vấn đề sống còn: tài liệu yêu cầu cho hệ thống kiểm soát động cơ máy bay chứa hàng nghìn câu yêu cầu, và phần lớn trong số đó mơ hồ đến mức hai kỹ sư khác nhau đọc cùng một câu có thể implement theo hai cách hoàn toàn khác nhau.

Hệ quả không phải là bug — là thảm họa hàng không. Họ cần một ngôn ngữ đặc tả có cấu trúc, dễ học, và buộc người viết phải tường minh về điều kiện, hành động, và hệ thống. EARS ra đời từ nhu cầu đó: Easy Approach to Requirements Syntax.

Mấy chục năm sau, EARS tìm được ứng dụng hoàn hảo thứ hai: viết Executable Spec cho AI Agent. Lý do giống hệt — AI không khoan nhượng với sự mơ hồ, giống như một máy bay không khoan nhượng với lỗi kỹ thuật. Nếu câu yêu cầu không rõ ràng, AI sẽ chọn một cách diễn giải tùy ý — và bạn không biết nó đã chọn cái gì.

📌 **EARS — Định nghĩa** Easy Approach to Requirements Syntax: bộ 5 mẫu câu có cấu trúc cố định để viết yêu cầu phần mềm loại bỏ hoàn toàn sự mơ hồ về điều kiện kích hoạt, chủ thể, và hành động. Mỗi mẫu trả lời: "WHEN nào? WHO/WHAT? SHALL làm gì?" Từ khóa SHALL là bắt buộc — phân biệt với SHOULD (khuyến nghị) hay MAY (tùy chọn).

5.3.1 — Pattern 1: Ubiquitous (Luôn luôn đúng)

Dùng cho các yêu cầu không có điều kiện — không trigger, không trạng thái, luôn áp dụng mọi lúc. Đây là hành vi nền tảng của hệ thống. Mỗi mẫu EARS được thiết kế cho một loại yêu cầu khác nhau. Việc chọn sai mẫu không chỉ là lỗi văn phong — nó che giấu thông tin quan trọng mà AI cần để implement đúng.

Cú pháp: THE <system> SHALL <action>.

Pattern 1: Ubiquitous — Không có điều kiện, luôn áp dụng

5.3.1 — Pattern 2: Event-driven (Khi sự kiện xảy ra)

Dùng khi hành vi được kích hoạt bởi một sự kiện cụ thể — hành động của người dùng, signal từ hệ thống, hay thay đổi trạng thái. Khác với Ubiquitous: đây là hành vi xảy ra TẠI MỘT THỜI ĐIỂM cụ thể, không phải liên tục.

Cú pháp: WHEN <event>, THE <system> SHALL <action>.

Pattern 2: Event-driven — Khi sự kiện kích hoạt

5.3.1 — Pattern 3: State-driven (Khi đang ở trạng thái)

Dùng khi hành vi phụ thuộc vào trạng thái hiện tại của hệ thống hoặc đối tượng. Khác với Event-driven: đây là hành vi liên tục trong khi trạng thái tồn tại, không chỉ tại thời điểm chuyển đổi. Đây là điểm khác biệt quan trọng — WHILE vs WHEN.

Cú pháp: WHILE <state>, THE <system> SHALL <action>.

Pattern 3: State-driven — Trong khi trạng thái tồn tại

5.3.1 — Pattern 4 & 5: Optional Feature + Unwanted

Pattern 4 — Optional Feature (Khi tính năng được bật)

Dùng cho các tính năng configurable — hành vi chỉ áp dụng khi feature flag, cài đặt, hoặc điều kiện hệ thống nhất định được kích hoạt. Rất phổ biến trong SaaS với nhiều tier.

Cú pháp: WHERE <feature> IS ENABLED, THE <sys> SHALL <action>.

5.3.2 — EARS Cheat Sheet · Tham khảo nhanh

Đây là bảng tham khảo nhanh để dùng ngay khi viết spec. Laminate và dán lên màn hình, hoặc thêm vào snippet của editor.

Mẫu	Cú pháp	Khi nào dùng	Ví dụ nhanh
Ubiquitous	THE <sys> SHALL <action>.	Luôn đúng, mọi lúc	THE sys SHALL hash passwords với bcrypt ≥12.
Event-driven	WHEN <event>, THE <sys> SHALL <action>.	Phản ứng với sự kiện	WHEN user clicks "Submit", THE sys SHALL validate trong 1s.
State-driven	WHILE <state>, THE <sys> SHALL <action>.	Hành vi liên tục trong trạng thái	WHILE offline, THE app SHALL queue writes locally.
Optional	WHERE <feature> IS ENABLED, THE <sys> SHALL <action>.	Feature flag / gói dịch vụ	WHERE 2FA enabled, THE sys SHALL require OTP after login.
Unwanted ★	WHERE <error/condition>, THE <sys> SHALL <response>.	Xử lý lỗi & edge cases	WHERE API fails 3×, THE sys SHALL refund và alert Slack.

⚠️ ★ **Unwanted là mẫu quan trọng nhất** AI sẽ implement phần còn lại theo cách nó thấy "hợp lý" — điều đó không kiểm soát được. Chỉ có Unwanted patterns mới buộc AI tư duy về failure modes.

5.3.3 — Phân cấp nghĩa vụ · SHALL / SHOULD / MAY

Đây là chi tiết nhỏ nhưng cực kỳ quan trọng. Khi AI gặp SHALL, nó hiểu đây là mandatory — không implement là bug. Khi gặp SHOULD, nó có thể skip nếu thấy phức tạp. Khi gặp MAY, nó có thể implement hoặc không, tùy "cảm hứng".

Keyword	Nghĩa	AI behavior	Khi nào dùng
SHALL	Bắt buộc — không thể bỏ qua	Implement 100%, thiếu = bug	Mọi yêu cầu core business logic
SHALL NOT	Bắt buộc KHÔNG làm	Không implement, nếu làm = bug	Security constraints, prohibitions
SHOULD	Khuyến nghị — tốt nếu có	Có thể skip khi khó hoặc conflicting	Best practices, non-critical quality
SHOULD NOT	Không khuyến nghị	Tránh nhưng không phải lỗi nếu vi phạm	Deprecated patterns, anti-patterns
MAY	Tùy chọn — được phép nếu muốn	Implement hoặc không, tùy context	Extensions, nice-to-have features

Ví dụ phân cấp trong cùng một spec (Password Reset)
WHEN user reset password, THE hệ thống SHALL gửi email có link hiệu lực 1 giờ.
THE reset link SHALL NOT hoạt động sau khi đã được sử dụng (one-time use).
THE hệ thống SHOULD gửi thêm SMS nếu user đã đăng ký số điện thoại.
THE email MAY bao gồm thông tin thiết bị và vị trí của request.

5.3.4 — Kết hợp 5 mẫu EARS · Full Login Spec

Một tính năng thực tế cần nhiều mẫu EARS kết hợp. Ví dụ sau mô tả toàn bộ tính năng "Đăng nhập" với tất cả 5 mẫu, bao gồm cả các edge case thường bị bỏ qua:

SPEC: User Authentication — Login Flow

Version: 1.2.0 | Owner: @security-team | Status: APPROVED

Ubiquitous

THE hệ thống SHALL không lưu password dưới dạng plaintext.

THE hệ thống SHALL không tiết lộ trong error message rằng username tồn tại hay không (prevent user enumeration).

Event-driven

WHEN người dùng submit form đăng nhập, THE hệ thống SHALL xác thực credentials trong vòng 2 giây.

WHEN xác thực thành công, THE hệ thống SHALL:

Tạo session token (JWT, TTL=30 phút),

ghi login_event {user_id, timestamp, ip, user_agent},

redirect đến dashboard hoặc intended_url.

State-driven

WHILE tài khoản bị khoá (locked=true), THE hệ thống SHALL từ chối mọi đăng nhập và hiển thị: "Tài khoản bị khoá. Liên hệ support@company.com."

Optional Feature

WHERE tài khoản bật 2FA, THE hệ thống SHALL redirect đến màn hình nhập OTP sau khi xác thực password. OTP hiệu lực 5 phút. Tối đa 3 lần nhập sai.

Unwanted (xử lý lỗi)

WHERE đăng nhập thất bại lần thứ 5 liên tiếp trong 15 phút, THE hệ thống SHALL:

(1) Khoá tài khoản 1h, (2) Gửi email cảnh báo,

(3) Ghi security_event {type:"brute_force", user_id, ip}.

WHERE session token hết hạn hoặc không hợp lệ, THE hệ thống SHALL redirect về login với query param ?reason=session_expired.

WHERE request đăng nhập từ IP ngoài whitelist (Enterprise only), THE hệ thống SHALL từ chối và ghi ip_blocked_event.

5.3.5 — Bài tập thực hành EARS

Bài tập 5.3.A ★

Phân loại mẫu EARS phù hợp và viết lại theo đúng cú pháp:

1. "If the user is logged in, show the dashboard."
2. "Error messages should be user-friendly."
3. "Premium users can export data as CSV."
4. "The system should handle network timeouts."
5. "Data is encrypted in transit."

📌 **Gợi ý đáp án 5.3.A** (1) State-driven: WHILE user logged in → (2) Ubiquitous: THE system SHALL → (3) Optional: WHERE Premium plan IS ENABLED → (4) Unwanted: WHERE network timeout occurs → (5) Ubiquitous: THE system SHALL encrypt data in transit with TLS 1.3.

Bài tập 5.3.B ★★☆☆

Viết Executable Spec đầy đủ cho tính năng "Quên mật khẩu" sử dụng tất cả 5 mẫu EARS.

Yêu cầu tối thiểu:

- Ít nhất 3 Unwanted patterns
- Phân cấp SHALL/SHOULD/MAY rõ ràng
- Kèm Out of Scope cho sprint
- Version + owner + status header

Bài tập 5.3.C ★★★★★

Đưa spec bài 5.3.B vào Cline với prompt:

Review spec này và liệt kê: (1) Các edge cases còn thiếu (2) Yêu cầu nào mơ hồ cần làm rõ (3) Contradiction nếu có

Format: numbered list. Không suggest giải pháp — chỉ chỉ ra vấn đề.

Ghi lại phản hồi của AI và update spec.

5.4

Levels of Specification Depth

Đúng mức · Đúng chỗ · Risk × Complexity

		LIKELIHOOD				
		RARE	UNLIKELY	POSSIBLE	LIKELY	ALMOST CERTAIN
IMPACT	INSIGNIFICANT	Lanp	Minor	Tesak Sasto	Lanly Focun	Acet Asopreter gal
	MINOR	Manat Cras	Pospere Cavi	Pisbu	Kel Kulgor	Lared base
	MODERATE	Ese		Poteet Sast	Bis Nepuds	Powam base
	MAJOR	Maske	Suphe Nony	Boyet Lony	Onset-lypuro	Kard Sack
	CATASTROPHIC	Laze	Fau's Novins 7	Boyet Dande	Invet Snyghe	Pence Pual the

5.4.1 — Risk x Complexity Matrix

Một sai lầm phổ biến của những người mới học SDD là áp dụng cùng một mức độ chi tiết cho mọi thứ. Họ viết Formal Spec đầy đủ 8 thành phần cho một hàm tiện ích 5 dòng code, và đồng thời chỉ viết một comment mơ hồ cho luồng thanh toán xử lý hàng triệu đồng. Cả hai đều sai.

Spec là đầu tư. Giống như mọi khoản đầu tư, ROI phụ thuộc vào việc đầu tư đúng chỗ. Mục tiêu là calibrate: viết spec đủ để AI thực thi đúng, không phải nhiều nhất có thể.

Risk \ Logic	Logic Đơn giản	Logic Vừa	Logic Phức tạp
Rủi ro Thấp (Bug = annoyance)	● Sketch — Prompt ngắn OK	● Detailed Spec 4–5 thành phần	● Detailed Spec 6–7 thành phần
Rủi ro Vừa (Bug = mất UX/data)	● Detailed Spec ngắn + test cases	● Detailed Spec đầy đủ 8 thành phần	● Formal Spec + State Diagram
Rủi ro Cao (Bug = tiền/bảo mật)	● Detailed Spec + security review	● Formal Spec + Diagram + Review	● Formal Spec + Diagram + Audit trail

Mức 1 — Sketch
Prompt có cấu trúc · < 3 branches · Rủi ro thấp · 5-10 phút viết

Mức 2 — Detailed
6–8 thành phần · EARS notation · 70% công việc thực tế · 30-60 phút

Mức 3 — Formal
8 thành phần + State Diagram + Security review · Thanh toán, auth · 2-4 giờ

5.4.2 Mức 1 — Sketch (Phác thảo) & 5.4.3 Mức 2 — Detailed (Chi tiết đầy đủ)

Mức 1 — Sketch (Phác thảo)

Sketch là mức tối thiểu — một prompt có cấu trúc. Phù hợp cho utility functions, helper methods, và các đoạn code nhỏ không có business logic phức tạp.

Khi nào dùng: Logic đơn giản (1–2 branch), rủi ro thấp, có thể kiểm tra output bằng mắt trong < 30 giây.

```
# Sketch — formatCurrency(amount, currency)
Input: amount (number), currency (ISO 4217)
Output: string đã format

VND: 1234567 → "1.234.567 đ"
USD: 1234.5 → "$1,234.50" (2 decimal)

Edge cases:
- amount < 0: throw Error("non-negative")
- unsupported: throw Error("Unsupported: X")

# Đây là đủ — không cần EARS, không cần State Diagram
```

Mức 2 — Detailed (Chi tiết đầy đủ)

Detailed Spec là mức tiêu chuẩn cho hầu hết các module tính năng. Bao gồm 6–8 thành phần, EARS notation, và Out of Scope rõ ràng. Phù hợp cho khoảng 70% công việc development thực tế.

Khi nào dùng: Module tính năng có 3–10 business rules, rủi ro vừa, cần unit test để verify.

```
# Detailed — ProductReview Module (tóm tắt)
## 3. Functional (EARS)
WHEN buyer submit review, THE hệ thống SHALL:
- Validate: rating 1–5, comment 10–500 chars
- Check: buyer đã mua (order.status=delivered)
- Check: chưa có review cho order này

WHERE buyer chưa mua, THE hệ thống SHALL ẩn form
+ hiển thị "Mua sản phẩm để đánh giá."

WHERE review có từ ngữ blacklist, THE hệ thống SHALL từ chối + log security event.
```

5.4.4 — Mức 3: Formal Spec + State Diagram

Formal Spec dành cho các luồng có rủi ro cao: thanh toán, xác thực, quản lý quyền. State Diagram buộc bạn liệt kê MỌI trạng thái có thể — kể cả các trạng thái trung gian mà prose spec thường bỏ qua. Với AI, State Diagram đặc biệt có giá trị: nó là nguồn duy nhất để AI biết transition nào là hợp lệ, transition nào không.

```
# Formal Spec — Payment Order Flow | v2.1.0 | @payment-team | Legal: 2025-01-10

## STATE DIAGRAM
[created] — [awaiting_payment]
      |
      +-----+
      |         |         |
      ▼         ▼         ▼
[payment_pending] [failed] [cancelled]
      |         |
      ▼         ▼
[payment_confirmed] [expired] (sau 30 phút)
      |
      ▼
[processing] — [shipped] — [delivered]
      |
      ▼
[refunded]
```

VALID TRANSITIONS (explicit — AI chỉ implement những transition này)

created → awaiting_payment: khi user confirm giỏ hàng

awaiting_payment → payment_pending: khi gọi payment gateway

payment_pending → payment_confirmed: khi webhook success

payment_pending → failed: khi webhook failure

INVALID TRANSITIONS (explicit prohibition)

AI SHALL NOT implement: delivered → cancelled (phải qua refunded)

AI SHALL NOT implement: failed → processing (phải thanh toán lại)

INVARIANTS

THE hệ thống SHALL:

(1) Tổng tiền order KHÔNG đổi sau created

(2) Mọi transition có audit_log entry

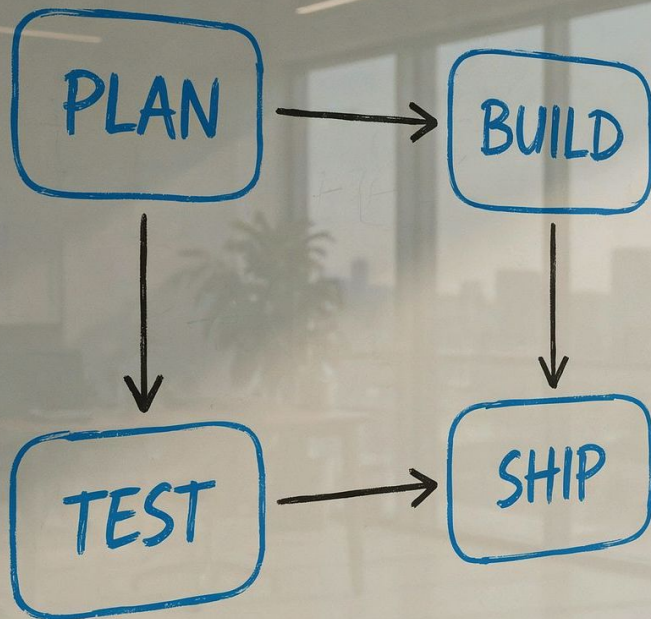
(3) Chỉ 1 active payment attempt tại một thời điểm

5.4.5 — Hướng dẫn quyết định nhanh

Khi bắt đầu bất kỳ task nào, hãy đặt 3 câu hỏi: (1) Nếu AI implement sai, hậu quả là gì? (2) Có bao nhiêu branch và edge case? (3) Có ai khác cần đọc và depend vào spec này không?

Module ví dụ	Rủi ro	Logic	Mức Spec đúng	Lý do
formatDate() helper	Thấp	Đơn giản	Sketch	Bug dễ thấy, fix nhanh
Search/filter products	Thấp	Vừa	Detailed	Nhiều filter combination
User profile update	Vừa	Vừa	Detailed	Data integrity + audit
Login/Auth flow	Cao	Vừa	Formal	Security, brute-force protection
Payment processing	Cao	Phức tạp	Formal	Tiền + state machine phức tạp
Notification email	Thấp	Đơn giản	Sketch	Dễ kiểm tra output
RBAC/Permissions	Cao	Phức tạp	Formal	Security + many role combos
Dashboard charts	Thấp	Vừa	Detailed	Nhiều data conditions

 **5.4.6 Bài tập — Calibrating Spec Depth** Cho e-commerce app: phân loại mức Spec cho: (1) Hàm tính shipping fee theo trọng lượng+tỉnh thành, (2) Module quản lý coupon nhiều điều kiện, (3) Hàm crop/resize ảnh, (4) Luồng KYC xác thực danh tính người bán, (5) Module email marketing hàng loạt, (6) Function generate unique order ID.



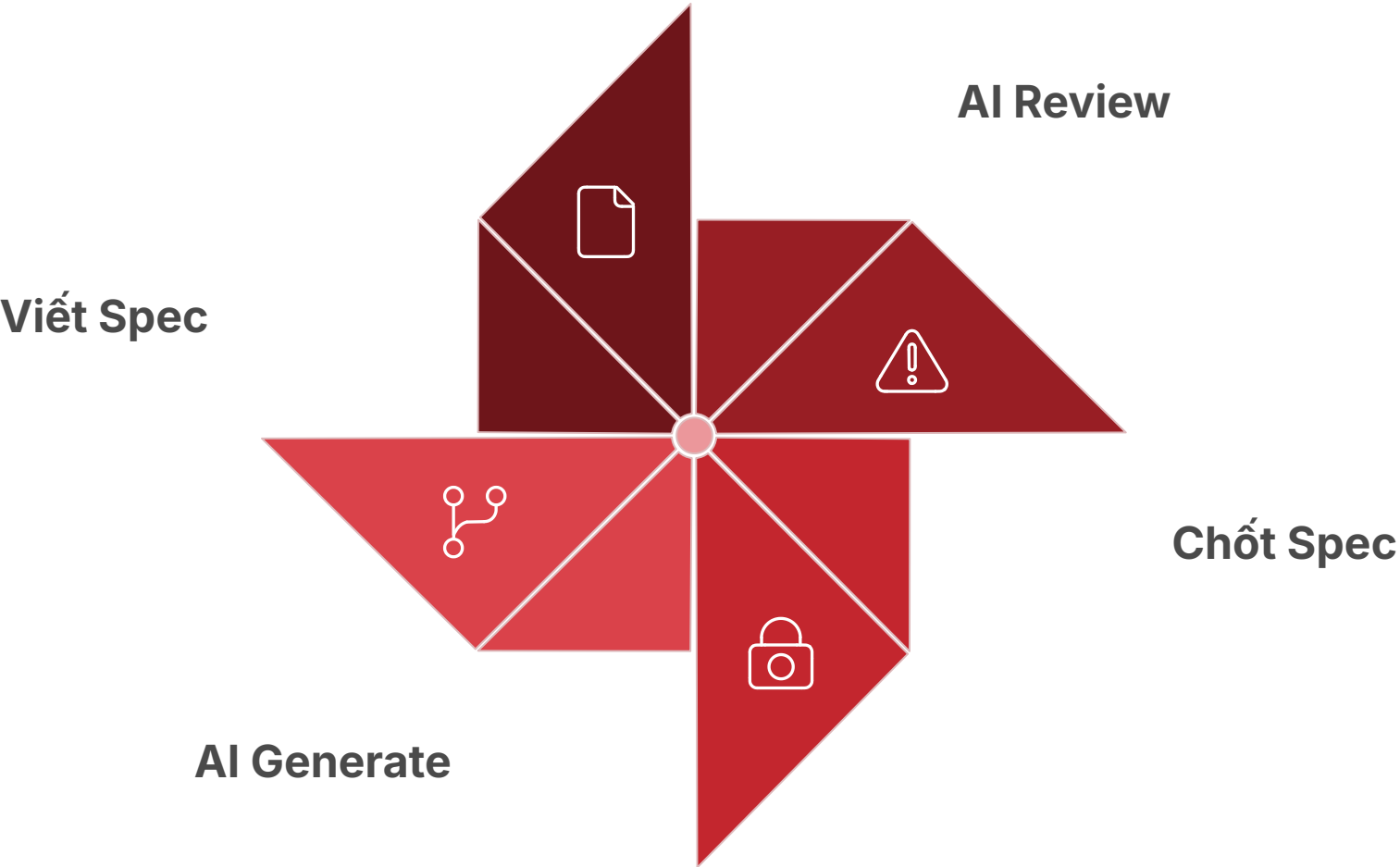
5.5

The Spec-Driven Development Workflow

Lập trình ở tầng ý định · 4 bước thực tế

5.5.1 — Quy trình 4 bước · Tổng quan

Phần này là điểm hội tụ của toàn bộ chương. Quy trình SDD không phải là "write spec → paste vào AI → nhận code". Đó là một vòng lặp cộng tác giữa người và máy, trong đó mỗi bên đóng góp điều mà mình làm tốt nhất: con người mang ý định, kinh nghiệm, và judgment; AI mang khả năng phân tích cú pháp, tìm logic gap, và thực thi không mệt mỏi.



5.5.2 — Bước 1 · Con người viết Spec sơ thảo

Bước này không có AI tham gia — và đây là chủ đích. Trước khi đưa AI vào, con người phải tự suy nghĩ về bài toán. Nhiều team mắc sai lầm bỏ qua bước này — họ đi thẳng từ "có ý tưởng" đến "chat với AI". Kết quả là spec được AI "đoán" từ một mô tả mơ hồ, và toàn bộ hệ thống được xây trên nền tảng không vững.

Story Decomposition — 5 câu hỏi trước khi viết

- 1. **Actor là ai?** Mọi loại người dùng, role, system khác sẽ interact với feature này.
- 2. **Happy path là gì?** Mô tả kịch bản thành công lý tưởng trong 2–3 câu.
- 3. **What could go wrong?** Liệt kê 5–10 điều có thể sai. Đây sẽ là Unwanted patterns.
- 4. **Ranh giới là gì?** Những gì KHÔNG thuộc scope — sẽ là Out of Scope.
- 5. **Khi nào "xong"?** Acceptance criteria — cách verify implementation là đúng.

SPEC.md Template khởi đầu nhanh

```
# [Feature Name] Spec
# Version: 0.1 (DRAFT) | Owner: @name

## 1. Context & Goal

## 2. Actors & Roles
- [Actor 1]: [quyền]

## 3. Functional Requirements

## 4. Non-functional Requirements

## 5. Data Model
## 6. Error Handling
## 7. Acceptance Criteria
- [ ] [criterion]

## 8. Out of Scope
- Không có...

## Notes / Open Questions
```

5.5.3 — Bước 2 · AI review Spec, tìm logic gaps

Đây là bước nhiều người không biết đến nhưng có impact lớn nhất. Thay vì dùng AI để viết code ngay, hãy dùng AI để "peer review" spec của bạn. AI rất giỏi tìm ra những điều bạn bỏ qua vì quá quen thuộc với domain — những assumption ẩn, edge case bị quên, và logic inconsistency.

Prompt mẫu cho AI Spec Review

Tôi cần bạn review spec này với tư cách là một kỹ sư senior.
Đừng viết code. Chỉ phân tích spec và trả lời 4 câu hỏi:

1. LOGIC GAPS: Có trạng thái/kịch bản nào spec chưa xử lý?
Liệt kê cụ thể: "Nếu [điều kiện X], hệ thống làm gì?"
2. CONTRADICTIONS: Có yêu cầu nào mâu thuẫn?
Trích dẫn cụ thể 2 yêu cầu conflicting.
3. MISSING EDGE CASES: Liệt kê 5 edge case quan trọng nhất chưa được đề cập. Ưu tiên: security, data integrity, concurrent access.
4. AMBIGUOUS REQUIREMENTS: Yêu cầu nào mơ hồ?
Liệt kê và hỏi làm rõ.

Format: numbered list, 1–3 câu/issue.
Không suggest giải pháp — chỉ chỉ ra vấn đề.

--- SPEC BẮT ĐẦU ---
[Paste SPEC.md ở đây]
--- SPEC KẾT THÚC ---

AI Review output mẫu — ProductReview

Logic Gaps (3 tìm thấy)

1. Spec không xử lý: Nếu buyer đặt 2 order cho cùng 1 SP và cả 2 đã delivered — buyer được phép tạo mấy review?
2. Thiếu: WHEN buyer EDIT review, THE system SHALL...
Spec cho phép sửa nhưng không có behavior khi edit.
3. Concurrent: Double-click Submit → 2 review đồng thời.
Idempotency chưa được spec.

Missing Edge Cases (5)

1. Buyer bị ban — review còn hiển thị?
2. Product bị xóa — reviews xử lý thế nào?
3. Admin xóa review — thông báo buyer không?
4. Rating = null vs rating = 0 trong DB?
5. Boundary max: 501 ký tự comment bị reject?

Ambiguous (2)

1. "Rating aggregate 5 phút" — eventual OK?
Hay read-after-write consistency?
2. Blacklist — ai maintain? AI detect hay list?

5.5.4 — Bước 3 · Con người chốt Spec (Source of Truth)

Sau khi nhận review từ AI, đây là lúc con người đưa ra judgment. Không phải mọi issue mà AI tìm thấy đều cần giải quyết ngay — đó là lý do tại sao bước này không thể tự động hóa. Con người phải quyết định: issue này quan trọng và cần xử lý trong spec, hay là acceptable risk và sẽ handle sau?

Decision framework cho từng AI-found issue

Loại issue	Action
Logic gap nghiêm trọng	Thêm vào spec NGAY
Edge case quan trọng	Thêm Unwanted pattern
Ambiguity có ảnh hưởng	Làm rõ trong spec
Out of scope (intentional)	Ghi vào Out of Scope
Nice to have, low risk	Ghi vào backlog
False positive của AI	Ignore + comment lý do

Lock ceremony — Cam kết với Spec

```
# Khi spec đã được approve
## 1. Update header
# Version: 1.0.0 ← Từ 0.x lên 1.x = locked
# Status: APPROVED ← Không còn là DRAFT
# Approved by: @lead-dev, @product-owner
# Locked on: 2025-01-20
# Sprint: Sprint-15 (ends 2025-02-03)
# RULE: Không modify trong sprint.
# Phát sinh → SPEC_addendum.md

## 2. Git commit với tag
git add SPEC.md
git commit -m "spec(review): approve v1.0.0"
git tag "spec/product-review/v1.0.0"

## 3. Slack message
# "
```

5.5.5 — Bước 4 · AI generate Code + Unit Tests

Bước này là nơi AI thực sự tỏa sáng — nhưng chỉ khi spec ở các bước trước đã làm tốt. AI được yêu cầu implement đồng thời code và unit test. Hai thứ này phải được generate cùng nhau để đảm bảo test thực sự cover spec, không phải chỉ cover code.

Prompt Code + Test generation

Implement feature theo SPEC.md đính kèm.

Tech stack: Python 3.12, FastAPI, pytest.

Yêu cầu output:

1. IMPLEMENTATION FILE: src/reviews/service.py

- Implement đầy đủ theo tất cả SHALL
- Tuân thủ mọi SHALL NOT constraints
- Error handling theo Unwanted patterns
- Comment EARS tag cho mỗi business rule:
EARS[Event]: WHEN user submits...

2. TEST FILE: tests/reviews/test_service.py

- Ít nhất 1 test per Acceptance Criteria
- Test names: test_[criterion_description]
- Bao gồm: happy path, errors, boundaries
- pytest fixtures, không hardcode data

3. TRACEABILITY MATRIX trong test file:

TEST → SPEC MAPPING

test_buyer_can_review → Section 3, bullet 1

Nếu có yêu cầu nào không chắc cách implement,
DỪNG LẠI và hỏi — đừng assume.

--- SPEC BẮT ĐẦU ---

[SPEC.md v1.0.0 APPROVED]

--- SPEC KẾT THÚC ---

EARS Tags trong source code

src/reviews/service.py — với EARS tags

```
async def create_review(self, buyer_id, order_id,  
product_id, rating, comment):
```

EARS[Ubiquitous]: Validate rating range

```
if not 1 <= rating <= 5:
```

```
raise ValidationError("Rating must be 1-5")
```

EARS[Ubiquitous]: Validate comment length

```
if not 10 <= len(comment) <= 500:
```

```
raise ValidationError("Comment 10-500 chars")
```

EARS[Unwanted]: WHERE buyer has not purchased

```
order = await self.order_repo.get(order_id)
```

```
if order.status != OrderStatus.DELIVERED:
```

```
raise ForbiddenError("Order must be delivered")
```

EARS[Unwanted]: WHERE review already exists

```
existing = await self.review_repo.find(order_id)
```

```
if existing:
```

```
raise ConflictError("Review already exists")
```

Happy path — tạo review

```
return await self.review_repo.create(review)
```

Traceability: search "EARS[" để tìm khi spec đổi

Review Checklist sau khi AI generate

Trước khi approve code do AI generate, reviewer cần verify đầy đủ những điểm sau. Đây là trách nhiệm con người — không thể outsource cho AI tự review chính nó.

Kiểm tra	Cách verify	Fail nếu...
Mọi SHALL được implement	So sánh spec với code line by line	Có SHALL không có code tương ứng
Mọi SHALL NOT được tuân thủ	Grep code cho forbidden patterns	Tìm thấy hành vi bị cấm
Mọi Acceptance Criteria có test	Map test names với AC list	Có AC không có test cover
Unwanted patterns được handle	Check error handling code	Exception bị swallow/ignore
Out of Scope được tôn trọng	Review nếu AI code thêm gì	Code nằm ngoài spec tồn tại
EARS tags present	Search for # EARS[	Nhiều business rule không có tag

❏ **"Lập trình ở tầng ý định"** Khi bạn viết SPEC.md, bạn không viết tài liệu — bạn đang "lập trình". Spec là chương trình. AI là compiler. Code là bytecode. Đây là sự dịch chuyển vai trò quan trọng nhất của AI era: Developer không còn là người "gõ code" — họ là người "thiết kế ý định".

Bài Tập Calibrating Spec Depth

Risk × Complexity · Shopping Cart Case

5.4.6 Bài tập — Calibrating Spec Depth

Đây là bài tập tổng hợp kết thúc chương. Tính năng Giỏ hàng cho e-commerce app — tính năng phổ biến nhưng có đủ độ phức tạp để thực hành đầy đủ: nhiều actors, nhiều state, edge cases thú vị, và non-trivial business rules.

- ❏ **Context để bắt đầu** User type: Guest (chưa đăng nhập) và Authenticated user · Cart items: product_id, variant_id (optional), quantity, unit_price · Inventory: có thể hết hàng bất cứ lúc nào · Price: có thể thay đổi sau khi đã add vào cart · Promotion: coupon code có thể apply vào cart · Guest cart: cần merge với user cart khi đăng nhập.

Phase 1 — Bạn tự viết Spec sơ thảo (45 phút)

Viết SPEC.md theo mẫu 8 thành phần. Trả lời 5 câu Story Decomposition trước khi viết.

Gợi ý Unwanted patterns:

- WHERE sản phẩm hết hàng sau khi đã add vào cart...
- WHERE giá sản phẩm thay đổi sau khi đã add...
- WHERE cùng 1 sản phẩm được add 2 lần...
- WHERE quantity > stock_available...
- WHERE coupon code hết hạn hoặc đã dùng...
- WHERE guest cart và user cart có cùng SP (merge conflict)...

Phase 2 — AI Review (15 phút với Cline)

Mở Cline trong VSCode. Paste spec sơ thảo và dùng prompt review mẫu từ Section 5.5.3. Ghi lại:

- AI tìm được bao nhiêu logic gaps?
- AI tìm được bao nhiêu edge cases bạn bỏ qua?
- Có contradiction nào không?
- AI hỏi làm rõ điều gì?

Phase 3 — Update & Lock (20 phút) Dựa trên review của AI, update spec và resolve mọi issue. Lock spec với version 1.0.0. Commit vào git với proper message.

Phase 4 — Generate Code + Tests (với Cline)

Dùng prompt từ Section 5.5.5 để yêu cầu Cline generate đầy đủ:

File	Path	Nội dung
File 1	src/cart/service.py	CartService với đầy đủ business logic + EARS tags
File 2	src/cart/models.py	SQLAlchemy models cho Cart, CartItem
File 3	src/cart/router.py	FastAPI endpoints: GET/POST/PUT/DELETE cart
File 4	tests/cart/test_service.py	Unit tests với traceability matrix

Phase 5 — Evaluation Rubric

Tiêu chí	Điểm tối đa	Cách đánh giá
Tất cả SHALL được implement	30đ	1đ/requirement, missing = -2đ
Unwanted patterns được handle	25đ	2đ/error case có handler đúng
Unit tests cover Acceptance Criteria	20đ	1đ/acceptance criterion có test
EARS tags trong code	10đ	5đ nếu ≥ 80% rules có tag
Out of Scope được tôn trọng	10đ	0đ nếu AI code ngoài scope
Code quality (readable, typed)	5đ	Subjective: type hints + naming

Tổng kết Chương 5

Reflection questions

- 1. Spec sơ thảo của bạn thiếu bao nhiêu edge cases trước khi AI review? "Blind spots" nói lên điều gì?
- 2. Chất lượng code AI generate khi có spec đầy đủ so với khi chỉ có mô tả ngắn khác nhau như thế nào?
- 3. Khi bạn viết Spec, bạn đang "lập trình ở tầng ý định" — vai trò developer thay đổi như thế nào?
- 4. Phần nào của quy trình SDD tốn nhiều thời gian nhất? Phần nào có thể tối ưu thêm?

Tổng kết — Tools & Concepts

Khái niệm	Điểm cốt lõi	Công cụ
Spec = Interface	Tách "Cái gì" khỏi "Như thế nào"	SPEC.md template
8 Thành phần	Out of Scope quan trọng như In Scope	8-component checklist
EARS Notation	5 patterns loại bỏ mơ hồ	EARS Cheat Sheet
Levels of Depth	Calibrate theo Risk × Complexity	Risk Matrix
SDD Workflow	Draft → Review → Lock → Generate	SPEC.md lifecycle
Anti-patterns	5 "tử huyệt" khiến AI hallucinate	Pre-spec checklist

❏ **Mindset shift quan trọng nhất** Bạn không còn là người "gõ code" hay "chat với AI". Bạn là người "thiết kế ý định" — người sở hữu spec chính là người sở hữu sản phẩm. Kỹ năng viết Executable Spec là kỹ năng quan trọng nhất của developer trong thời đại AI. Chương 6 sẽ đào sâu hơn: Test-Driven Specification — BDD Given-When-Then, test suite trước code.